Teach yourself GraphQL in 2026

an anti-blueprint

Jeff Auriemma

Why we're here

🤔

Why we're here

We want to make...

- better user experiences
- better apps
- better agents

GraphQL is the investment to make in 2026

I need your help

The dream query

```
query FlightSearch {
  flights(origin: "JFK", destination: "LHR", date: "2026-06-01")
{
    airline
    departureTime
    arrivalTime
    legs {
      airport
      duration
    }
    price {
      amount
      currency
    }
  }
}
```

A GraphQL query that contains an ideal representation of the data needed to create a full user experience

Query anatomy

```
query FlightSearch {
  flights(origin: "JFK") {
    airline
    legs {
      airport
      duration
    }
  }
}
```

operation type — query, mutation, subscription

operation name — optional, but always use one

field — a named piece of data

argument — parameterizes a field

selection set — { } — describes the shape you want back

nesting — fields can contain fields

The response mirrors the shape of the query.

Introducing myself

```
query TalkIntro {
  me {
    name
    title
    company
    bio
    socials {
      github
      bluesky
      linkedin
    }
  }
}
```

GraphQL is

GraphQL is

... just a spec

:(

GraphQL is

... just a spec

:)

GraphQL is

- "just" a spec
- a means of expressing your domain objects
- a way to get new experiences, apps, and agents off the ground quickly

Learn GraphQL in 2026

an anti-blueprint

This talk...

- is spontaneous and social
- focuses on basics and questions, not complexity and answers
- uses a prompt-driven narrative to match modern dev practices

Make the dream a reality

Schema

```
type Query {
  flights(origin: String!, destination: String!, date: String!): [Flight!]!
}

type Flight {
  airline: String!
  departureTime: String!
  arrivalTime: String!
  legs: [Leg!]!
  price: Price
}

type Leg {
  airport: String!
  duration: Int!
}

type Price {
  amount: Float!
  currency: String!
}
```

type — a named object in your domain

field — a typed piece of data on a type

scalar — leaf type — String, Int, Float, Boolean, ID

! — non-null — the field will always have a value

[ ] — list

arguments — typed inputs declared on a field

Every field selected in the query has a definition here. The schema defines what's possible; the query picks what it wants.

Descriptions

```
"A scheduled airline flight between two airports."
type Flight {
  "Two-letter IATA code, e.g. `AA`, `BA`."
  airline: String!

  "ISO 8601 timestamp in the origin airport's local time."
  departureTime: String!

  """
  All segments of the journey, in order.
  A direct flight has exactly one leg.
  """
  legs: [Leg!]!

  "Null when pricing isn't available for this passenger."
  price: Price
}
```

`"..."` — single-line description

`"""..."""` — multi-line, markdown-friendly

Lives on types, fields, arguments, enum values — anywhere in the schema

Introspectable — IDEs, codegen, docs, and agents all read it

Lintable: enforce descriptions to make sure your API is well-annotated

In 2026 your schema is read by humans and agents. A well-described schema is a well-prompted agent. Special directives such as `@deprecated` further enrich schemas.

Resolvers

```ruby
class Types::QueryType < Types::BaseObject
  field :flights, [Types::FlightType], null: false do
    argument :origin, String, required: true
    argument :destination, String, required: true
    argument :date, String, required: true
  end

  def flights(origin:, destination:, date:)
    HTTP.get(
      "http://service-1/flights",
      params: { origin:, destination:, date: }
    ).parse
  end
end
```

resolver — a function that returns data for a field

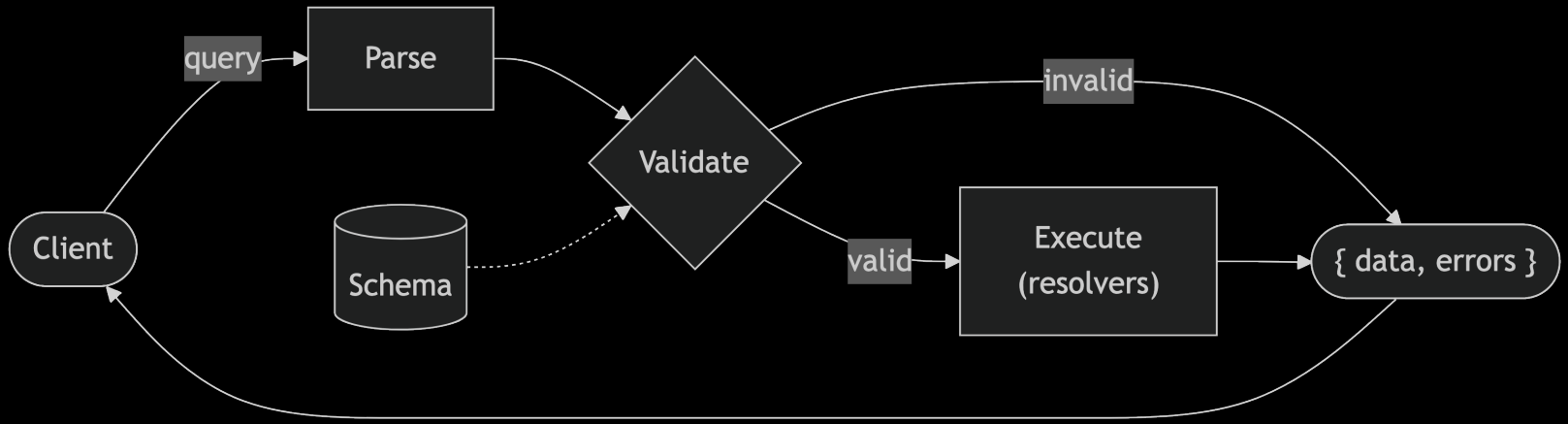
field — declared once in the schema, implemented once as a method

arguments — arrive as keyword arguments, already typed and validated

body — plain runtime code (Ruby in this case) — call a database, a REST API, another service

The schema says what flights returns. The resolver says how to get it. GraphQL doesn't care what's underneath.

Anatomy of a GraphQL request



Validate before execute — bad queries never reach your resolvers.

Query shape == response shape — the client picks; the server delivers exactly that.

Changing data with mutations

Mutations

```
mutation BookFlight($flightId: ID!, $passengerId: ID!) {
  bookFlight(
    flightId: $flightId,
    passengerId: $passengerId
  ) {
    booking {
      id
      status
    }
  }
}
```

- Same structure as a query — operation type, name, selection set
- For writes: creates, updates, deletes
- Returns data too — no need for a follow-up request

Resolving mutations

```
func (r *mutationResolver) BookFlight(
    ctx context.Context,
    flightID, passengerID string,
) (*BookFlightPayload, error) {
    body, _ := json.Marshal(map[string]string{
        "flightId":    flightID,
        "passengerId": passengerID,
    })
    resp, err := http.Post("http://service-2/bookings", "application/json",
bytes.NewReader(body))
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()

    var booking Booking
    json.NewDecoder(resp.Body).Decode(&booking)
    return &BookFlightPayload{Booking: &booking}, nil
}
```

- Not the same as HTTP verbs
- Still resolvers — plain code, any language (here: Go + gqlgen)
- Same wiring as a query resolver — just write semantics
- Return the new state so the client doesn't need a follow-up read

Variables

Parameterize an operation — separate the document from the runtime values.

```
mutation BookFlight($flightId: ID!, $passengerId: ID!) {
  bookFlight(
    flightId: $flightId,
    passengerId: $passengerId
  ) {
    booking {
      id
      status
    }
  }
}
```

```
{
  "flightId": "AA42",
  "passengerId": "u_8821"
}
```

`$variableName: Type` — declared at the top, passed separately at runtime.

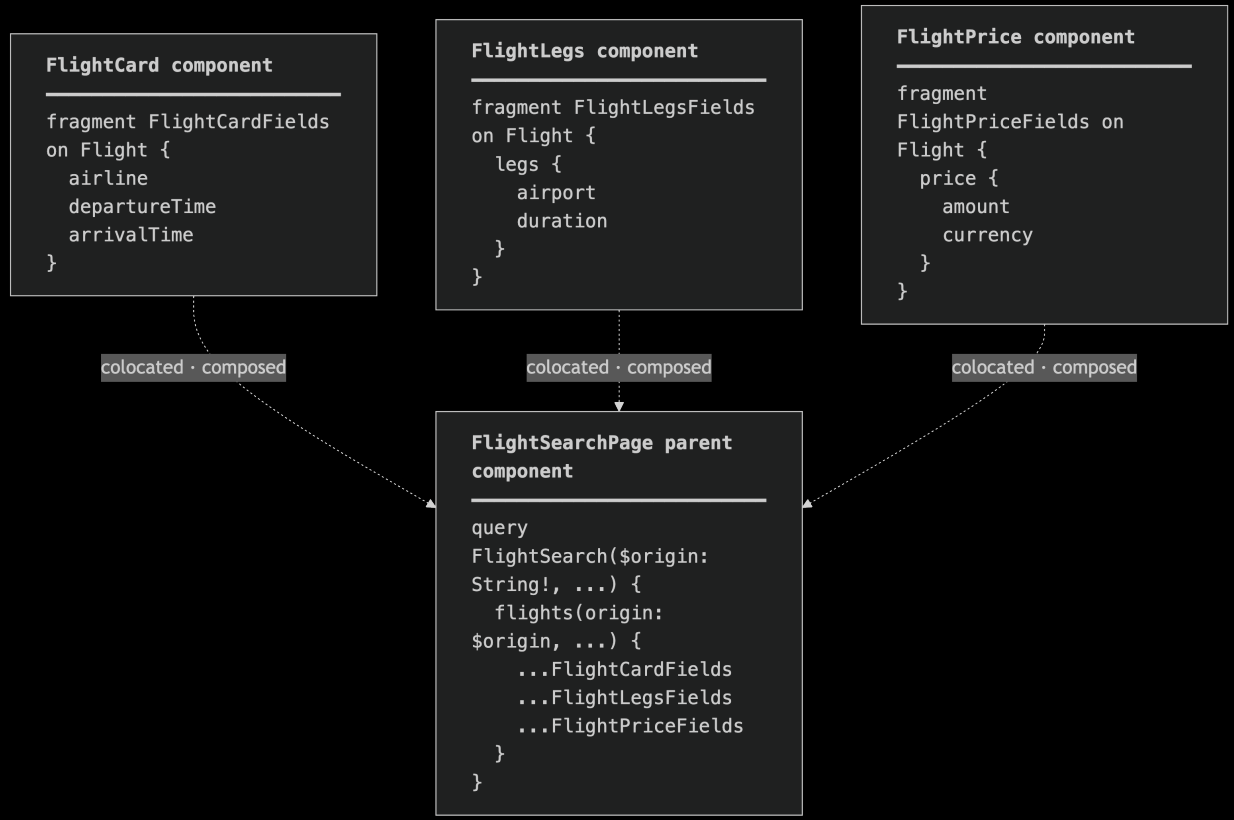
Learning through schema design

With your agent...

1. Try to describe your data
2. Determine the best way to interact with your systems
3. Collaborate before committing
4. Some prefer schema-first, others code-first

Questions you'll be prompting later

How should I use fragments?



How is GraphQL typesafe for $MY_LANGUAGE?

```graphql
# FlightSearch.graphql
query FlightSearch($origin: String!) {
  flights(origin: $origin) {
    airline
    departureTime
    legs {
      airport
      duration
    }
  }
}
```

```swift
// FlightSearchQuery.swift  (generated)
class FlightSearchQuery: GraphQLQuery {
  let origin: String

  struct Data: SelectionSet {
    let flights: [Flight]

    struct Flight: SelectionSet {
      let airline: String
      let departureTime: String
      let legs: [Leg]

      struct Leg: SelectionSet {
        let airport: String
        let duration: Int
      }
    }
  }
}
```

Code generation tools read your GraphQL operations + the schema and emit strongly-typed code (in this case: Swift). Change the query, regenerate, the compiler catches every broken consumer.

How is GraphQL typesafe for $MY_LANGUAGE?

Caution

Some additions may not be backwards compatible with clients. Even enum additions!

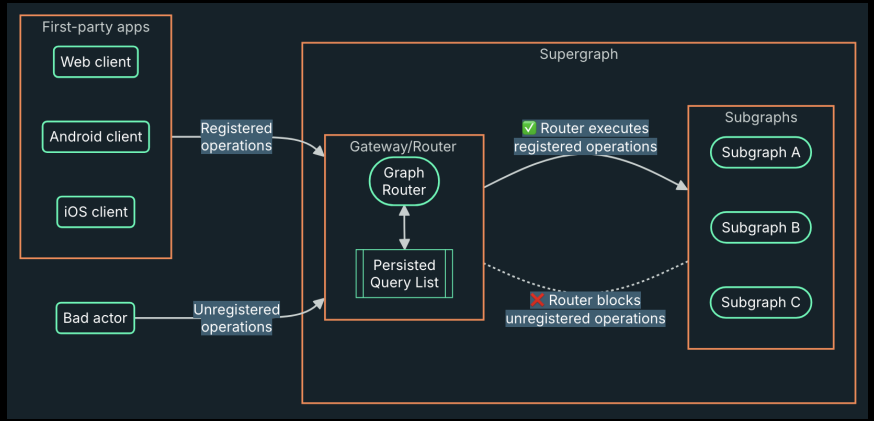
```
# V1
enum Example {
  FOO
  BAR
}
```

```
# V2
enum Example {
  FOO
  BAR
  BAZ
}
```

How should I secure my deployment?

- Persisted queries / trusted documents
- Rate/depth limiting
- No introspection in production

How should I secure my deployment?



Persisted Queries / Trusted Documents architecture

What techniques are good for performance and stability?

Questions you'll be prompting later

- Persisted queries / trusted documents
- DataLoader (N+1 problem)
- Client-side caching
- Server/router-side caching
- Incremental delivery
- Traffic shaping

How do I practice context engineering with GraphQL?

Questions you'll be prompting later

- OSS GraphQL MCP Servers are out there
- OSS Agent Skills for GraphQL are available
- Semantic Introspection is an exciting development
- MCP Tools for various GraphQL tools

Steal these slides

Repo

Resources

- GraphQL.org/learn
- Apollo Odyssey
- https://graphql.org/community/training-courses/

Thanks!

- GraphQL Foundation and TSC
- Apidays / FOST / Mehdi for partnering
- Apollo for funding my travel and lodging here
- Everyone here today for making this a reality